

CWG3103 “Corresponding members and by-value object parameters”

Document #: P4138R2 [[Latest](#)] [[Status](#)]
Date: 2026-04-20
Project: Programming Language C++
Audience: Evolution Working Group
Reply-to: Vlad Serebrennikov
<serebrennikov.vladislav@gmail.com>

Contents

- [1 Abstract](#)
- [2 Revision history](#)
- [3 Background](#)
- [4 Status quo](#)
- [5 Not all !\[\]\(38441ceaa711016e0bf2ad46ad394ff4_img.jpg\) are created equal](#)
- [6 Proposed approach](#)
- [7 Proposed wording](#)
 - [7.1 Wording option 1](#)
 - [7.2 Wording option 2](#)
 - [7.3 Annex C entry](#)
- [8 References](#)

1 Abstract

While looking at [CWG3103](#), I got interested how we arrived at status quo, specifically how [\[basic.scope.scope\]/3.1](#) came to be, and this is what I found. One of the conclusions I arrived at is that it doesn't make much sense to overload member functions with explicit object parameter of non-reference type with member functions of any other kind of object parameter with the same type, ignoring references.

2 Revision history

R2:

- Add discussion of move-only types and address of an overload set to [Not all !\[\]\(95b42f0077faf7439a26242a54e021ec_img.jpg\) are created equal](#).

R1:

- Minor changes to the wording after CWG review on 2026-03-24.

R0:

- Initial revision.

3 Background

The intent to give member functions with no *ref-qualifier* special treatment can be tracked all the way to [\[N1821\] \(Extending Move Semantics To *this \(Revision 2\)\)](#), which introduced *ref-qualifier*:

The `&` and `&&` suffixes are allowed only on members that allow a cv-qualifier in the same location (e.g., not constructors!). Additionally, neither of these forms can be overloaded with the existing form (the latter retains its binding semantics, including the exception that allows rvalues to be bound).

I think this was caused by the fact that implicit object parameter of a member function with no qualifiers can bind even rvalues, despite being of non-const lvalue reference type ([\[over.match.funcs.general\]/5](#)), which, I think, means the identity conversion and leads to ambiguous calls even without this restriction.

This, and the obvious rule that object parameters of the same type correspond, shape the status quo of the wording.

4 Status quo

The table below summarizes whether object parameters of two declarations of member functions correspond () or don't correspond (). Relevant wording is [\[basic.scope.scope\]/3](#) and [\[over.match.funcs.general\]/4](#).

	<code>()</code>	<code>()</code> <code>&</code>	<code>()</code> <code>&&</code>	<code>(this</code> <code>D)</code>	<code>(this</code> <code>D&)</code>	<code>(this</code> <code>D&&)</code>
<code>()</code>						

	<code>()</code>	<code>() &</code>	<code>() &&</code>	<code>(this D)</code>	<code>(this D&)</code>	<code>(this D&&)</code>
<code>() &</code>	✗	✗				
<code>() &&</code>	✗	✓	✗			
<code>(this D)</code>	✗	✓	✓	✗		
<code>(this D&)</code>	✗	✗	✓	✓	✗	
<code>(this D&&)</code>	✗	✓	✗	✓	✓	✗

Implementations agree on 18 out of 21 cases ([Compiler Explorer](#)):

- GCC and MSVC fully conform;
- EDG has a bug where explicitness of the object parameter by itself is enough to break correspondence despite types being the same;
- Clang sometimes considers `(this)` + `(this D&)` and `(this)` + `() &` cases to correspond, but this might be a desirable direction as discussed in section. This happens when they are written in exactly this lexical order (this quirk has been filed as [llvm/llvm-project#189525](#)).

However, when one of the declarations is named by a using-declaration, implementations agree only on 18 out of 36 cases, which includes 15 additional symmetrical cases left blank in the table ([Compiler Explorer](#)):

- MSVC struggles a lot;
- EDG struggles a lot, too, but less than MSVC;
- Clang conforms if using-declaration and declaration of member function are in the “right” lexical order, which is a unique behavior across surveyed implementations ([llvm/llvm-project#187686](#));
- GCC fully conforms.

Note that in this latter case correspondence (✗) doesn’t translate to program being ill-formed per [\[namespace.udecl\]/11](#).

5 Not all are created equal

The table below summarizes which value category the object argument has to have for a call to be well-formed, focusing on cases with `(this)` overload.

	By itself (copy+move)	With <code>(this CopyMove)</code>	By itself (move- only)	With <code>(this MoveOnly)</code>
1. <code>()</code>	Both	Conflicting	Both	Conflicting

	By itself (copy+move)	With (this CopyMove)	By itself (move- only)	With (this MoveOnly)
2. () &	Lvalue	Rvalue	Lvalue	Rvalue
3. () &&	Rvalue	Lvalue	Rvalue	Neither
4. (this)	Both	Conflicting	Rvalue	Conflicting
5. (this &)	Lvalue	Rvalue	Lvalue	Rvalue
6. (this &&)	Rvalue	Lvalue	Rvalue	Neither

All implementations [agree](#), with an exception of Clang, which considers more cases conflicting, as described in the previous section.

Note how adding a (**this**) overload to an existing member function F either flips the set of well-formed calls to its complement, rendering F obsolete, because (**this**) is always the best viable function, or outright make any call ill-formed (cases 3 and 6 with move-only type). However, a function that cannot be selected by an overload resolution for a call expression can still be selected by an address of an overload set with the right target ([Compiler Explorer](#)):

```
struct MoveOnly {
    MoveOnly();
    MoveOnly(MoveOnly&) = delete;
    MoveOnly(MoveOnly&&);

    void f() &;           // #1
    void f(this MoveOnly); // #2

    void g(this MoveOnly&&); // #3
    void g(this MoveOnly);   // #4
};

void test(MoveOnly& moveonly) {
    moveonly.f();           // ambiguous
    MoveOnly{}.f();         // selects #2
    void(MoveOnly::*p1)() &= &MoveOnly::f; // selects #1

    moveonly.g();           // selects #4, then fails
    MoveOnly{}.g();         // ambiguous
    void(*p2)(MoveOnly&&) = &MoveOnly::g;   // selects #3
}
```

While it's possible to get ahold of functions #1 and #3 (via address of an overload set) and call them, it's not clear why they need to be a part of their respective overload sets in the first place.

6 Proposed approach

Explicit object parameter of non-reference types should correspond with any other object parameter if they have equivalent types, ignoring references, i.e. it should be ill-formed to have overloads with such object parameters that don't have any other differences. This change will have the following effects:

- Correspondence of member functions with explicit object parameter of non-reference type will be synchronized with implicit member functions with no *ref-qualifier*.
- The second example in [CWG3103](#) will become ill-formed as requested.
- Clang rejecting `(this) + (this D&)` and `(this) + () &` overloads when they are written in exactly this lexical order, as described above, will become conformant behavior.

Table below summarized correspondence of object parameters with the proposed changes applied. Combinations that are made to correspond by this change are marked as **XXX**.

	<code>()</code>	<code>() &</code>	<code>() &&</code>	<code>(this D)</code>	<code>(this D&)</code>	<code>(this D&&)</code>
<code>()</code>	✗					
<code>() &</code>	✗	✗				
<code>() &&</code>	✗	✓	✗			
<code>(this D)</code>	✗	✗✗	✗✗	✗		
<code>(this D&)</code>	✗	✗	✓	✗✗	✗	
<code>(this D&&)</code>	✗	✓	✗	✗✗	✓	✗

There is a perspective on this approach where it restores pre-C++23 status quo: you either write a single member function with no *ref-qualifier* (that match any kind of object argument), or you're an advanced user and you write one or two functions with *ref-qualifier* that do exactly what you need. No mixing is allowed.

7 Proposed wording

Hide removals: ☐

Strikethrough: ☒

§ 7.1 Wording option 1

Change 6.4.1 [\[basic.scope.scope\]](#) paragraph 3 as follows:

3

Two non-static member functions have *corresponding object parameters* if

- (3.1) — ~~exactly one is an implicit object member function with no *ref-qualifier* and the types of their object parameters (9.3.4.6 [dcl.fct]), after removing references, are the same, or~~
- (3.1) — the types of their object parameters (9.3.4.6 [dcl.fct], 12.2.2 [over.match.funcs]) are equivalent~~;~~ or
- (3.2) — if one of the functions is
 - (3.2.1) — an implicit object member function with no *ref-qualifier* or
 - (3.2.2) — a member function with an explicit object parameter whose type is not a reference typethe types of their object parameters, after removing references, are equivalent.

§ 7.2 Wording option 2

Change 6.4.1 [basic.scope.scope] paragraph 3 as follows:

- 3 Two non-static member functions have *corresponding object parameters* if
- (3.1) — exactly one is an implicit object member function with no *ref-qualifier* and the types of their object parameters (9.3.4.6 [dcl.fct]), after removing references, are the same, or
 - (3.2) — one is a member function with an explicit object parameter whose type is not a reference type, and the types of their object parameters, after removing references, are equivalent, or
 - (3.3) — the types of their object parameters are equivalent.

§ 7.3 Annex C entry

Add a new subclause to C.1 [diff.cpp23] for 6 [basic] with the following paragraph:

Affected subclause: 6.4.1 [basic.scope.scope]

Change: Explicit object parameter of non-reference type corresponds with any other object parameter of the equivalent type, ignoring references.

Rationale: If an overload set of member functions that differ only in their object parameter has a function with an explicit object parameter of non-reference type, other functions in the overload set cannot be selected by overload resolution.

Effect on original feature: A valid C++ 2023 program that declares such an overload set is ill-formed.

[Example:

```
struct S {  
    void f(this S);  
    void f() &;           // ill-formed; previously well-formed  
    void g(this S);  
    void g() &&;          // ill-formed; previously well-formed  
    void h(this S);  
    void h(this S&;       // ill-formed; previously well-formed  
    void i(this S);  
    void i(this S&&);     // ill-formed; previously well-formed  
};  
  
— end example]
```

8 References

[N1821] D. Vandevorde, B. Kozicki. 2005-08-24. Extending Move Semantics To `*this` (Revision 2).

<https://wg21.link/n1821>